RADAR

Receivables Risk Scoring & Credit Decision Support Platform

Technical Documentation · v1.0

DynamicAI · 2026

Radar — A-to-Z Technical Documentation

🎯 Customizable for Your Company

Radar is an open-source (AGPL-3.0) platform and is **customizable to each company's own business rules, sector profile and risk appetite**. Scoring formulas, weights, sector risk factors, macro assumptions (interest, inflation), payment-term tiers and decision logic are fully configurable.

Contact DynamicAI for **enterprise integration, custom module development, on-premise deployment and training**: *osmanedembele6361@gmail.com*

> Receivables risk scoring and credit decision support platform. Last updated: 2026-05-03

---

1. Project Overview

Radar is a **credit-scoring / receivables-risk evaluation** application aimed at SMEs and corporate firms. It is positioned as the in-house (private) counterpart of a Findeks-like rating service.

Core workflow: 1. The user creates a customer (account) 2. Enters aging and credit-request data, or imports it in bulk from Excel 3. The system produces a **credit grade** based on payment discipline, financial ratios (Z-Score, F-Score, DSCR, ICR), sector risk factors and macro conditions 4. Monte Carlo simulation + named stress tests compute the scenario distribution 5. A multilingual (TR/EN/ES/DE) detailed PDF/HTML report is produced

---

2. Technology Stack

Layer	Technology
Backend	Python 3.11, **Flask 3.0** (Blueprints)
ORM	**SQLAlchemy 2.0** (scoped_session)
DB	PostgreSQL (prod) / SQLite (dev) — dual target
Auth	flask-login + **Authlib** (Google OAuth) + itsdangerous (mail verify)
Mail	flask-mail (SMTP)
i18n	**flask-babel 4.x** (locale_selector pattern)
Templating	Jinja2 + Bootstrap 5.3 + Bootstrap Icons
API Docs	**Flasgger** (OpenAPI/Swagger)
Async	Celery + Redis (planned, not yet active)
Container	Docker + docker-compose, `network_mode: host`
WSGI	gunicorn 4 workers
Reverse Proxy	nginx — `technodai.com/radar/` → `127.0.0.1:5001`
Excel	pandas + openpyxl

3. High-Level Architecture

```
┌──────────────────────────────────────────────────────┐
│  Browser (TR/EN/ES/DE)                               │
└──────────────────────────────────────────────────────┘
                        │ https://technodai.com/radar
                        ▼
                  ┌──────────┐
                  │  nginx   │  TLS, prefix routing
                  └──────────┘
                       │ proxy_pass :5001
                       ▼
              ┌──────────────────┐
              │  gunicorn × 4    │  Docker (host net)
              │   ↓              │
              │  Flask app       │
              │   ├ blueprints/  │  auth, main, customer,
              │   │              │  scoring, admin, api
              │   ├ enterprise/  │  i18n, RBAC, audit, webhooks
              │   └ analytics/   │
              └──────────────────┘
                        │
                        │
            ┌───────────┴───────────┐
            ▼          ▼          ▼
       PostgreSQL   SMTP      Google OAuth
       (advisory   (verify     (login)
        locks)     mails)
```

---

4. Module Map

4.1 Core

File	Responsibility
`app.py`	Flask factory (`create_app`), blueprint registration, `init_db()` , enterprise feature init
`database.py`	SQLAlchemy models, dual-DB engine, lightweight ALTER migration, **Postgres advisory lock** (concurrent worker safety)
`extensions.py`	`login_manager` , `mail` , `ts` (URLSafeTimedSerializer) — module-level shared singletons
`credit_scoring.py`	`CreditScorer` class — score calculation, expert assessment notes, Monte Carlo + named stress tests
`aging_analyzer.py`	Builds delay metrics from aging records
`excel_import.py`	Bulk customer/record import (pandas)
`currency.py`	TRY formatting, FX conversion helpers
`logger.py`	Structured logging
`translations.py`	4-language dictionary (TR/EN/ES/DE) — flat dict, ~70 KB

4.2 Routes (Blueprints)

Blueprint	Path	Responsibility
`auth`	`/login` , `/register` , `/logout` , `/google-login` , `/verify/<token>`	User authentication
`main`	`/` , `/nedir` , `/guvenlik` , `/set_language/<lang>`	Home, static content, language switch
`customer`	`/musteri/...`	CRUD + detail
`scoring`	`/skor/...` , `/rapor/<id>`	Score calculation, report render
`admin`	`/admin/...`	Settings, user management, audit log
`api`	`/api/v1/...`	REST endpoints (documented via Flasgger)

4.3 Enterprise Layer (Phase 8-11)

File	Responsibility
`enterprise.py`	`init_enterprise_features(app)` — RBAC, audit, multi-tenant init
`i18n_utils.py`	flask-babel 4.x locale selector
`analytics.py`	Tenant-level metric aggregation
`api_docs.py`	Flasgger Swagger UI configuration
`webhooks.py`	Outbound webhook delivery (Slack, Discord, custom)

---

5. Data Model

```
Tenant 1──* User 1──* Customer 1──* AgingRecord
                                  │
                                  ├──* CreditRequest
                                  └──* CreditScore (history)

User *──* Role  (UserRole pivot)
Tenant 1──* AuditLog
```

Important: - `tenant_id` is `nullable=True default=1` on every table — legacy rows fall to the default tenant - Customers with `is_sample=True` are visible to all users (demo data) - `CreditScore` is versioned — every calculation creates a new row, history is never deleted

---

6. Deployment Architecture

Prod (technodai.com)

```
~/apps/radar/                  # git clone
├── docker-compose.yml         # local-only port binding (5001)
├── .env                       # SECRET_KEY, DB_URL, OAuth, SMTP
└── data/                      # SQLite fallback + uploads (volume)
```

- Container `radar_v1_app`, `restart: always`, host network
- Postgres `radar` DB, `pg_advisory_lock(7382001)` prevents the 4-worker init race
- nginx vhost: `/radar/` prefix `proxy_pass http://127.0.0.1:5001/`

Local (dev)

- Same compose file (Desktop symlink), port `8005`
- Falls back to SQLite, runs without a separate DB

Deployment Flow

```
local edit
   ↓ git commit + push
   ↓ ssh prod
cd ~/apps/radar && git pull
sudo docker compose up -d --build
   ↓ smoke test
curl -I https://technodai.com/radar/
```

---

7. Development Workflow

1. **Branch**: small commits on `main`. Topic branches for larger features.
2. **Migration**: `database.py:_run_lightweight_migrations()` uses an `ALTER TABLE IF NOT EXISTS` pattern — Alembic intentionally avoided for the small schema.
3. **Seed**: `seed_sample_data.py` — demo customer set for fresh installs.
4. **Tests**: `tests/security_smoke.py` runs in CI; `integrity_check.py` is manual.
5. **i18n**: New strings must be added to all 4 blocks of `translations.py`. CI enforces key parity.
6. **Commit message**: `<type>(<scope>): <summary>` — `feat`, `fix`, `chore`, `refactor`.

---

8. End-User Flow

Step	Page	Action
1	`/register`	Email/password or Google sign-up → mail verification
2	`/`	Customer list — add new customer
3	`/musteri/yeni`	Account info (sector, equity, revenue, etc.)
4	`/musteri/<id>`	Add aging row + credit request
5	`/musteri/<id>/import`	Bulk import via Excel
6	`/skor/<id>`	Compute score — admin settings (interest, inflation) applied
7	`/rapor/<id>`	Detailed report — executive assessment, scenarios, named stress tests
8	`/admin`	Settings, user management, audit log

---

9. Roadmap — What's Next

9.1 Short Term (1-2 weeks)

- [ ] **Payment-term module expansion**: currently static tiered terms (`vade_15..90`); dynamic computation by payment-discipline band (DSO + sector median)
- [ ] **PDF export**: WeasyPrint integration for `?format=pdf` on the report
- [ ] **Unit-test scaffold**: `pytest` + `pytest-flask` , ≥70% coverage on `credit_scoring`
- [ ] **Audit log UI**: filtered table inside the admin panel

9.2 Medium Term (1-2 months)

- [ ] **Celery activation**: Excel import + score batch off the request thread (large files time out today)
- [ ] **Webhook event expansion**: `score.calculated` , `customer.high_risk` , `request.spike`
- [ ] **Rate limiting (per-tenant API)**: Redis-backed quota active for login/register; per-tenant `/api/v1/*` quota is open work
- [ ] **Sector benchmark feed**: public sector-ratio benchmark (KGK / TÜİK / Findeks) — currently peer median only
- [ ] **Notification channels**: email + Slack + Discord — high-risk alerts
- [ ] **Multi-currency**: everything is TRY today — FX snapshot table for USD/EUR accounts

9.3 Long Term (3+ months)

- [ ] **ML layer**: rule-based score + optional logistic regression / gradient boosting overlay (once enough records exist)
- [ ] **Open Banking**: account-movement pull → automatic aging
- [ ] **Mobile app**: PWA or React Native — read-focused (score view, alerts)
- [ ] **Self-serve tenant onboarding**: marketplace SaaS (today tenant creation is manual)
- [ ] **GDPR/KVKK panel**: data export, deletion request, retention policy
- [ ] **2FA**: TOTP (pyotp) — admin roles in particular
- [ ] **Audit log export**: JSON stream for SIEM (Splunk, ELK)

---

10. Continuous Improvement — Hardening Done

Topic	Improvement	Status
docker-compose `version` field obsolete	Field removed	✅
No webhook retry	`_post_with_retry` — 3 attempts + exponential backoff (1s/2s/4s); 5xx and network errors retry, 4xx single attempt	✅
No log-correlation ID	`request_id` middleware — UUID16 per request, `X-Request-ID` response header + `g.request_id` log binding	✅
Audit log was hard-deleted	`AuditLog.deleted_at` column — soft delete pattern (compliance retention)	✅
No DB backup automation	`scripts/backup.sh` — auto-detects Postgres/SQLite, gzip, retention rotation; **prod cron 03:00**	✅
Flask 3.0.0 CVE-2026-27205	`flask>=3.1.3` pin + pip/wheel/setuptools upgrade in CI	✅
Rate limiter `memory://` split across workers	`LIMITER_STORAGE_URI=redis://localhost:6379/1` — shared counter, verified in prod (11th wrong-pass returned 429)	✅
Executive Assessment frozen in TR when report reopened in another language	`assessment_i18n` / `decision_summary_i18n` — stored as a 4-language JSON dict; view falls to active language; legacy rows backwards compatible	✅
Excel import did not assign `user_id` to new `Customer` rows	`current_user.id` assignment — orphan-record issue resolved	✅
`Excel Upload` showed in menu for non-admin users (clicking redirected)	`context_processor` `is_admin` flag + `{% if is_admin %}` gating; redundant `Panel` link removed	✅
No alert on 5xx	`errorhandler(500)` → mail to `ADMIN_EMAIL` with request_id + path + user + exception summary	✅

Open (on the roadmap)

Topic	Impact	Plan
No Alembic	Lightweight ALTER is enough; revisit if schema grows	When the schema crosses 30+ tables
Low test coverage	Refactor risk	Pytest skeleton in place, target 50% on critical paths
`translations.py` is one 70 KB file	Merge-conflict magnet	One file per language + lazy load
Authlib deprecation (`authlib.jose`)	Will break on 2.0	Migrate to `joserfc`
Server-side override docker-compose	`git stash` on every pull	Split into `docker-compose.override.yml`

---

11. Security Notes — Active Controls

Identity & Session

- ✅ Passwords hashed with werkzeug PBKDF2-SHA256 (salted)
- ✅ Mail verification mandatory (URLSafeTimedSerializer, time-limited)
- ✅ Google OAuth — Authlib state validation (CSRF protection)
- ✅ Cookie flags: `HttpOnly` , `SameSite=Lax` , `Secure` in prod

Web Layer (flask-wtf, flask-limiter, flask-talisman)

- ✅ **CSRF protection**: `csrf_token` on every POST form; API blueprint exempt (token-based)
- ✅ **Rate limiting**: `/login` 10/min + 50/hour, `/register` 5/min + 20/hour; default 100/min + 1000/hour. **Prod backend Redis** (`LIMITER_STORAGE_URI=redis://localhost:6379/1`) — counter shared across workers.
- ✅ **Security headers** (Talisman): HSTS (1 year), `X-Frame-Options: SAMEORIGIN` , `X-Content-Type-Options: nosniff` , `Referrer-Policy: strict-origin-when-cross-origin`
- ✅ **Request ID**: every request carries an `X-Request-ID` header → log correlation
- ✅ **Menu admin gating**: `is_admin` context flag hides links requiring `@admin_required` from non-admin users

Data Access

- ✅ Multi-tenant isolation — every query filters by `tenant_id`
- ✅ SQLAlchemy ORM — parameterised queries (SQL injection protection)

- ✅ `is_sample` flag does not bypass tenant isolation
- ✅ Postgres advisory lock — concurrent init race protection
- ✅ Audit log soft-delete — no hard deletion, compliance retention

Operational

- ✅ Secrets in `.env`, never in repo (`.gitignore` + 8 security tests)
- ✅ DB backup script + retention rotation (`scripts/backup.sh`) — **prod cron every night at 03:00**
- ✅ Dependencies pinned, `pip-audit --strict` in CI (Flask CVE bump included)
- ✅ Upload cap 10 MB (DoS surface reduction)
- ✅ **5xx admin alert**: `errorhandler(500)` mails request_id + exception to `ADMIN_EMAIL`

Roadmap (Open)

- ⚠️ 2FA (TOTP) — admin roles in particular
- ⚠️ Webhook outbound HMAC signature mandatory by default
- ⚠️ CSP (Content Security Policy) — currently inline style/script, refactor needed
- ⚠️ Authlib `joserfc` migration (2.0 deprecation)

---

12. Operational Commands

```
# Local dev
python app.py                                    # SQLite

# Docker (local port 8005)
docker compose up -d --build
docker compose logs -f radar-app

# Prod deploy (from local machine)
sshpass -p "$SERVER_PASS" ssh "$SERVER_USER@$SERVER_IP" \
  "cd ~/apps/radar && git pull && \
   echo \$PASS | sudo -S docker compose up -d --build"

# Smoke test
curl -I https://technodai.com/radar/

# DB backup (on prod)
sudo docker exec radar_v1_app pg_dump -U radar radar > backup_$(date +%F).sql

# Migration (lightweight, runs automatically in init_db)
python -c "from database import init_db; init_db()"

# Seed
python seed_sample_data.py

# Translation key parity (manual)
python -c "from translations import translations; \
  ks = {l: set(translations[l]['expert_assessments'].keys()) for l in translations}
  print('diff:', ks['tr'] ^ ks['en'])"
```

13. Decision Records (mini-ADR)

1. **Lightweight ALTER instead of Alembic**: schema small, downtime acceptable. Revisit when complexity grows.
2. `network_mode: host` : same host as nginx, no port-mapping complexity. Will change for multi-host.
3. **flask-babel 4.x**: Flask 3.0 is incompatible with 3.x (`locked_cached_property` removed).
4. **Single tenant default ID=1**: legacy rows backfilled; multi-tenant transition was zero-downtime.
5. **Postgres advisory-lock key 7382001**: fixed magic number — all workers grab the same lock, the first creates the schema, the rest wait.
6. **Sponsor pill in the footer**: navbar lacked space, visibility was preserved by moving it to the footer.

14. License & Open Source

- **Project licence:** AGPL-3.0 — anyone hosting it as a service must share source
- **Dependencies:** all permissive (MIT/BSD/Apache); no GPL/AGPL → AGPL-compatible
- **Notable dependencies:**
- `psycopg2-binary` (LGPL) — dynamic linking, no issue
- `certifi` (MPL-2.0) — file-level copyleft, OK as long as we do not modify it
- **Licence audit:** done with `pip-licenses`, output clean

15. Test & Quality

Existing tests

- `tests/security_smoke.py` — 8 security checks:
- `.env` and secrets are not in git
- No private-key files
- No hardcoded SECRET/PASSWORD/API_KEY
- Critical packages are pinned (flask, sqlalchemy, authlib, itsdangerous, flasgger)
- All protected routes are wrapped with `@login_required`
- SECRET_KEY is set and long
- LICENSE file is AGPL-3.0
- `tests/integrity_check.py` — manual score validation (requires DB)

CI (GitHub Actions)

- `.github/workflows/ci.yml` — on every push/PR:
- pytest security suite
- `pip-audit --strict` dependency vulnerability scan
- Translation key parity (TR/EN/ES/DE)

Run

```
pytest tests/ -v                    # all tests
pytest -m security                  # security smoke only
pip-audit                           # CVE scan
```

16. References

- Repo: `git@github.com:Dai-Solutions/Radar.git`
- Prod: https://technodai.com/radar/
- Sponsor: OpenCollective + Patreon (`https://www.patreon.com/c/DynamicAI`)
- Findeks (competitor): https://www.findeks.com/